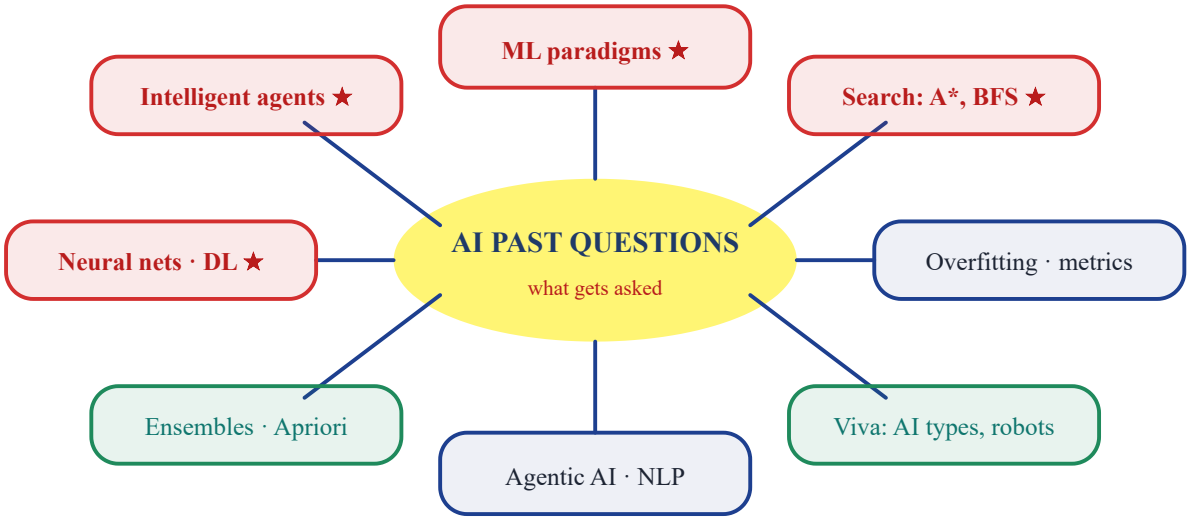
Solved Past Questions: Artificial Intelligence

30 questions from BCS, BUET-set, power-sector and bank papers and vivas, solved in 12 sheets



↑ red = asked most often (written + MCQ + viva together)

Papers covered

- **45th BCS Written (CSE-971)**, 31.01.2024
- **49th BCS (ICT-281)** MCQ paper, 2025: 9 AI MCQs
- BPDB AE (CSE) 24.02.2023 · GTCL AE (CSE) 2022
- DPDC AM (ICT) 27.06.2025 · BUET MSc in CSE 03.10.2026
- Bank Assistant Programmer written · bank IT vivas

Most asked

① Supervised vs unsupervised vs reinforcement (3 written)
② Agents: rational agent, 5 types
③ A* vs BFS, hill climbing
④ DL, backprop, RNN

Legend: **yellow** = key term · **red circle** = must-memorise · **purple** = memory trick · **red box** = exam trap · **green** = worked example · **blue box** = question as asked

Contents

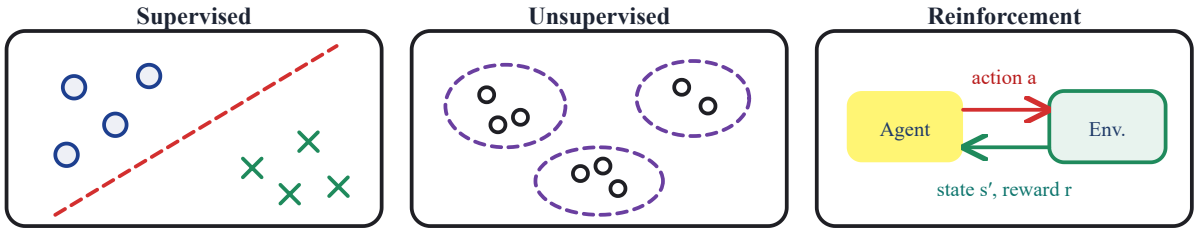
✓ ML: three paradigms …… p. 2
✓ Reinforcement & local search … p. 3
✓ Intelligent & rational agents … p. 4
✓ A* vs BFS …… p. 5
✓ Overfitting, ensembles …… p. 6
✓ Agentic AI vs chatbots …… p. 7
✓ Market basket: Apriori …… p. 8
✓ Deep learning, NLP …… p. 9
✓ MCQs asked + viva …… p. 10
✓ Night-before sheet …… p. 11
✓ Practice MCQs + written …… p. 12

1 Machine Learning: Three Paradigms

Q: What is machine learning? Differentiate among supervised, unsupervised and reinforcement learning. 45th BCS Written (CSE-971) · 31.01.2024; also in another written paper

Q: Compare and contrast the three fundamental paradigms of ML. 10 marks Bank Assistant Programmer written

Machine learning: The branch of AI in which a system **learns from data or experience** instead of being explicitly programmed (Samuel, **1959**). Mitchell: a program learns from experience **E** on task **T** with measure **P** if its performance on T, by P, **improves with E**.



labelled ○ / × → learn boundary | no labels → find clusters | trial and error → policy π

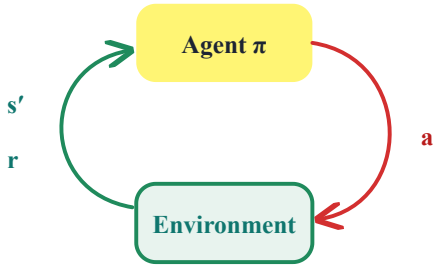
Aspect	Supervised	Unsupervised	Reinforcement
Data	**labelled** pairs (x, y)	unlabelled x only	experience (s, a, r, s′) from interaction
Feedback	correct answer per example	none	scalar **reward**, often delayed
Goal	learn f : x → y; predict	discover hidden structure	learn **policy π(s) → a** that maximises total reward
Tasks	classification, regression	clustering, PCA, association rules	sequential decisions, control
Algorithms	linear/logistic regression, decision tree, SVM, k-NN, Naive Bayes	k-means, hierarchical, DBSCAN, PCA, Apriori	Q-learning, SARSA, DQN, policy gradient
Evaluated by	accuracy, F1, RMSE on test data	silhouette, support/lift	cumulative reward per episode
Example	loan-default prediction, spam filter	customer segments, odd transactions	game playing, trading bot, robot control

Common ground: All three generalise from past data or experience to **unseen** cases. They differ in **the kind of feedback**: the full answer, none, or a score.

Memory trick: A **teacher** gives answers (supervised) · **no teacher**, find groups (unsupervised) · a **critic** gives only scores (reinforcement).

2 Reinforcement Learning: Policy, Reward, ε-Greedy

The pieces: **agent** · **environment** · state s · action a · reward r · **policy π** (state → action: the thing RL learns) · value V(s) and action-value Q(s, a) · discount γ ($0 \le \gamma \le 1$). Goal: maximise the return $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$



MCQ: _____ algorithm requires a policy. (supervised learning / fuzzy logic / deep learning / reinforcement learning) → **reinforcement learning** 49th BCS (ICT-281), 2025

Worked example: one Q-learning update ✍️: Rule: $\mathbf{Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]}$

Q(s, Right) = 2, reward r = 5, in s′ the best Q is 4, α = 0.5, γ = 0.9.

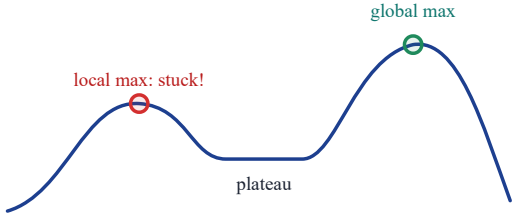
1. Target = r + γ·max Q(s′) = 5 + 0.9 × 4 = 8.6
2. TD error = 8.6 − 2 = 6.6
3. New Q(s, Right) = 2 + 0.5 × 6.6 = **5.3**

ε-greedy: With probability ε pick a **random** action (explore); otherwise the action with the highest Q (exploit). With ε = 0.1 and 4 actions the best action is chosen with probability 0.9 + 0.1/4 = **0.925**. ε is usually decayed over time.

- **Q-learning**: off-policy (learns the greedy policy)
- **SARSA**: on-policy (learns the policy it follows)
- **DQN**: a neural net approximates Q
- Uses: games (AlphaGo), robots, trading, ad bidding

Exam trap: RL is **not** unsupervised: the reward grades an action but never states the correct one. Pure exploitation (ε = 0) can lock onto a poor action forever.

3 Local Search: Hill Climbing & Random Search



hill climbing only moves uphill

Hill climbing moves to the best neighbour while it improves; O(1) memory. Drawback: **gets stuck in local optima**, plateaus and ridges. Fixes: random restarts, simulated annealing (accepts a worse move with probability $e^{-\Delta E/T}$), local beam search.

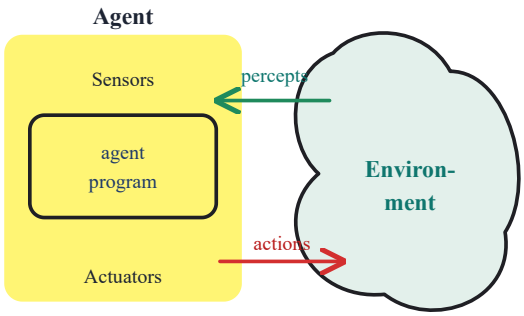
Random search samples candidates from the **whole** space and keeps the best one, so it is a **global optimisation** method: no gradient, cannot be trapped, but slow and never guaranteed optimal. Both MCQs: 49th BCS (ICT-281), 2025.

4 Intelligent & Rational Agents

Q: An artificial intelligence agent is an entity that continuously perceives its environment and acts upon it. Explain.
BPDB Assistant Engineer (CSE) · 24.02.2023

Agent: Anything that **perceives its environment through sensors** and **acts on it through actuators**. The agent function maps the percept sequence to an action; agent = **architecture + program**.

Rational agent: For every percept sequence it picks the action **expected to maximise its performance measure**, given its percepts and built-in knowledge. AI is the study of rational agents.



Rationality depends on: performance measure, prior knowledge, available actions, percept sequence. Rational ≠ **omniscient**: it maximises **expected**, not actual, outcome. Task description **PEAS**, e.g. automated taxi: P = safe, fast, legal trip · E = roads, traffic · A = steering, brake, horn · S = cameras, GPS, speedometer.

Properties: situated in an environment · **autonomy** (no direct human control) · **reactivity** (responds to change in time) · **pro-activeness** (takes initiative toward goals) · **social ability** (works with humans and agents) · learning.

Q: What are the types of agent in AI? Write their tasks. 5 marks 45th BCS Written (CSE-971) · 31.01.2024

Type	Chooses the action using …	Example	Limitation
① Simple reflex	current percept + **condition–action rules**	thermostat	fails if partially observable
② Model-based reflex	percept + **internal state** (how the world evolves, what my actions do)	car tracking cars in its blind spot	no explicit goal
③ Goal-based	model + **goal**; **search and planning**	GPS route planner	every goal state equally good
④ Utility-based	**utility function** U(state); maximises expected utility	taxi: speed vs safety vs fuel	utility hard to design
⑤ Learning	performance element + **learning element** + critic + problem generator	spam filter	needs data, exploration

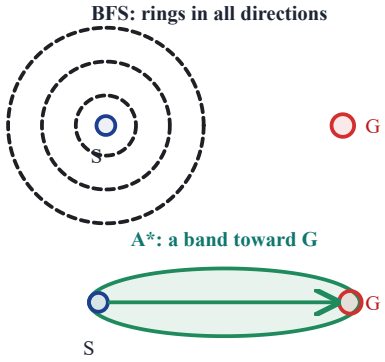
Memory trick: Each type adds **one** thing: reflex → + memory → + goal → + happiness score → + self-improvement.

Exam trap: Goal-based asks "**will** this reach the goal?"; utility-based asks "**how good** is this way?"

5 Heuristic (A*) vs Blind (BFS) Search

Q: Construct a logical argument explaining why a heuristic search (like A*) might be faster than a blind search (like BFS), even if it does not guarantee the absolute perfect path in all cases. 10 marks Bank Assistant Programmer written

1. **Cost = nodes expanded.** BFS generates $1 + b + b^2 + \ldots + b^d = O(b^d)$ nodes and stores them all (b = branching factor, d = goal depth).
2. **BFS is blind.** It does not know where the goal is, so it expands **every** node at depth k before depth k + 1: a ripple in all directions.
3. **A* is informed.** It expands the node with the smallest **f(n) = g(n) + h(n)** (g = cost so far, h = estimated cost to goal). Nodes leading away from the goal get a large f and are never expanded.



same S and G, far fewer nodes

4. **Smaller effective branching factor.** A good h makes b* close to 1, so work $\approx (b^*)^d$, not b^d. 8-puzzle, depth 12: blind iterative deepening ≈ **3.6 million** nodes; A* (misplaced tiles) 227; A* (Manhattan distance) **73**.
5. **Why the path may not be perfect.** A* is optimal only if h is **admissible** (never overestimates; for graph search also **consistent**: $h(n) \le c(n, n') + h(n')$). Practical systems use a stronger, inadmissible h, **weighted A*** ($f = g + w \cdot h$, $w > 1$, cost ≤ w × optimal) or greedy best-first (f = h): far fewer expansions, but the cheapest path may be missed.
6. **Conclusion.** The speed comes from the **information in h** (with h = 0, A* becomes uniform-cost search, i.e. BFS for equal step costs). Trading guaranteed optimality for guidance turns an exponential, unfocused search into a focused one; a near-optimal route in milliseconds (GPS, games, robots) beats a perfect one found too late.

	BFS	Uniform-cost	Greedy best-first	A*
Orders by	depth	g(n)	h(n)	g(n) + h(n)
Complete?	yes	yes	no (tree search)	yes
Optimal?	only for equal step costs	yes	**no**	yes, if h admissible
Memory	$O(b^d)$	exponential	exponential (worst)	keeps all nodes

Exam trap: "A* always finds the shortest path" holds **only** with an admissible h. Its worst case is still exponential and **memory** is its real limit (hence IDA*, SMA*).

Memory trick: **g** = "**g**one so far", **h** = "**h**ow far still". Admissible = an **optimist**: never says the goal is farther than it is.

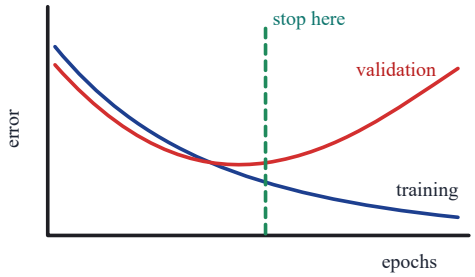
6 High Training, Low Test Accuracy

Q: What causes high training accuracy but low testing accuracy in a machine learning model, and what are its reasons and mitigation techniques? BUET MSc in CSE · 03.10.2026

Answer: overfitting: The model has **memorised the training set, noise included**, instead of the general pattern: low bias, **high variance**; the train–test difference is the generalisation gap.

Reasons

- model too **complex** (many parameters, deep tree)
- **too little** data, or too many features
- **noisy labels** and outliers get fitted
- training for **too many epochs**
- test data from a **different distribution**



Mitigation: ① **more data** / augmentation ② simpler model, feature selection, tree pruning ③ **regularisation**: L2 (loss + $\lambda\Sigma w^2$), L1, dropout ④ **early stopping** on validation loss ⑤ **k-fold cross-validation** ⑥ ensembles (bagging, random forest) ⑦ clean labels; same distribution for train and test. Underfitting is the opposite: low train **and** low test accuracy (high bias).

7 Weak vs Strong Learners (Ensembles)

Q: Explain weak and strong learners in ensemble learning. GTCL Assistant Engineer (CSE) · 2022

Weak learner: Only **slightly better than random guessing** (binary error just under 0.5), e.g. a **decision stump**.

Strong learner: Can reach **arbitrarily high accuracy** with enough data. Schapire (**1990**): weak learners can be **boosted** into a strong one.

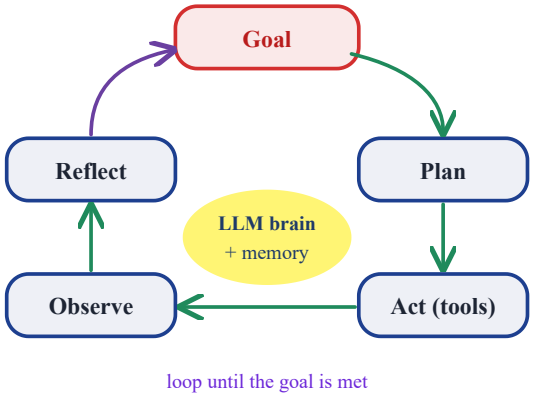
	Bagging	Boosting
Training	**parallel**, each on a bootstrap sample	**sequential**; misclassified rows get more weight
Combine	majority vote / average	weighted vote $H(x) = \text{sign}(\Sigma\ \alpha_t h_t(x))$
Reduces	**variance**	**bias** (and variance)
Examples	Random Forest	AdaBoost, Gradient Boosting, XGBoost

Worked example ✍️: Three independent classifiers, each 70% accurate, vote. P(majority right) = $0.7^3 + 3(0.7^2)(0.3) = 0.343 + 0.441 =$ **0.784** > 0.7. AdaBoost weight $\alpha_t = \frac{1}{2} \ln((1 - \varepsilon_t)/\varepsilon_t)$ is positive only while $\varepsilon_t < 0.5$.

8 Agentic AI vs Traditional Chatbots

Q: What is Agentive AI? Compare Agentive AI with traditional Chatbot AI. BUET MSc in CSE · 03.10.2026

Agentic (agentive) AI: An AI system given a **goal, not just a prompt**, which pursues it with little human supervision: it **plans** steps, **acts** through tools (APIs, search, code, databases, e-mail), **observes** results, keeps **memory** and **corrects itself** in a loop until the goal is met. It is usually built around a large language model (LLM).



Building blocks

- **Reasoning**: an LLM splits the goal into sub-tasks
- **Planning**: steps revised on failure (ReAct: reason → act → observe)
- **Tool use**: function calls to APIs, code, databases
- **Memory**: context + long-term (vector store)
- **Guardrails**: permissions, human approval, audit logs

A traditional chatbot is a **reactive question-answer** program: rule-based (pattern or intent matching, as in ELIZA, **1966**) or an LLM that only replies.

Point	Traditional chatbot	Agentic AI
Starts from	each user message	a **goal** (or an event)
Autonomy	none: the user drives every turn	**high**: chooses its next step
Planning	none	decomposes and re-plans
Actions	text replies only	**acts** in other systems via tools
Memory	current conversation at most	short- and long-term
Adapts	fixed scripts	checks results, self-corrects
Risk	a wrong answer	a wrong **action**: needs oversight
Bank example	answers "What is my balance?"	finds a failed transfer, reads logs, retries it, opens a ticket, informs the customer

Memory trick: A chatbot **talks**; an agent **does**. Agentic AI = a goal/utility-based **learning agent** (sheet 4) with an LLM as its program.

Exam trap: Agentic AI is **not** AGI, and not just "a bigger chatbot": the difference is **autonomy + tools + a goal loop**.

9 Market Basket Analysis: Apriori

Q: Which machine learning algorithm is suitable for market basket analysis? Explain the steps involved. DPDC Assistant Manager (ICT) · 27.06.2025

Answer: **Association-rule mining** with the **Apriori** algorithm (Agrawal & Srikant, **1994**), an **unsupervised** method (FP-Growth is the faster alternative). It finds rules such as {Diaper} → {Beer}: items bought together.

Measures: support(X) = count(X) / N · confidence(X → Y) = support(X ∪ Y) / support(X) · lift = confidence / support(Y), > 1 = real association.
Apriori property: every subset of a frequent itemset is frequent, so a candidate with an infrequent subset is pruned without counting.

1. Fix **min_support** and **min_confidence**.
2. Scan the data, count single items (C_1), keep those ≥ min_support → L_1.
3. **Join** L_{k-1} with itself → candidates C_k; **prune** those with an infrequent (k−1)-subset.
4. Scan, count C_k, keep ≥ min_support → L_k; repeat 3–4 until $L_k = \emptyset$.
5. From each frequent itemset form rules X → Y; keep confidence ≥ min_confidence, check lift.

Worked example ✍️: 5 baskets, min_support = 3 (60%), min_confidence = 70%.

T	Items
1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

C_1: Bread 4, Milk 4, Diaper 4, Beer 3, Cola 2 ✗, Eggs 1 ✗

C_2: {B,M} 3 · {B,D} 3 · {M,D} 3 · {D,Beer} 3 · {B,Beer} 2 ✗ · {M,Beer} 2 ✗

C_3: {B,D,Beer}, {M,D,Beer} pruned (infrequent subset); {B,M,D} = 2 ✗ → **$L_3 = \emptyset$, stop**

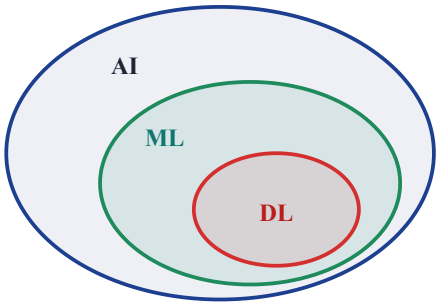
Rule	Support	Confidence	Lift	Verdict
Beer → Diaper	3/5	3/3 = 100%	1.0/0.8 = **1.25**	**strong ✓**
Diaper → Beer	3/5	3/4 = 75%	0.75/0.6 = **1.25**	**strong ✓**
Bread → Milk	3/5	3/4 = 75%	0.75/0.8 = 0.94	lift < 1: no real link

Exam trap: Confidence alone misleads: Bread → Milk has 75% confidence, but 80% of all baskets hold Milk anyway, so lift 0.94.

Memory trick: "**A rare set makes every bigger set rare**": that is the pruning. Uses: shelf layout, cross-selling, bank product bundles.

FP-Growth compresses the baskets into an **FP-tree** and mines it with only **2 database scans and no candidate generation**, so it is faster than Apriori on large data.

10 Deep Learning, Backpropagation, RNN



AI ⊃ ML ⊃ deep learning

Deep learning: The part of ML that uses **neural networks with many hidden layers**, which learn features straight from raw data.

	Classical ML	Deep learning
Features	hand-engineered	**learned automatically**
Data, compute	small–medium, CPU	large data, GPU
Explainability	easier	"black box"

Backpropagation: ① **forward pass**: output and loss L ② **backward pass**: the **chain rule** gives $\partial L/\partial w$ for every weight, output layer first ③ **update** by gradient descent $\mathbf{w \leftarrow w - \eta \cdot \partial L/\partial w}$ (η = learning rate). It trains MLPs, CNNs and RNNs (as backpropagation through time, BPTT).

Network	Built for	Key idea
MLP / ANN	tabular data	fully connected layers
CNN	images (spatial data)	convolution filters, pooling, shared weights
RNN / LSTM / GRU	**sequences, time series**, speech	hidden state carries the past: $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$
Transformer	language (LLMs)	self-attention over the whole sequence

Exam trap: Plain RNNs suffer **vanishing gradients** on long sequences; LSTM/GRU gates fix this. Gradient descent is an **optimisation method** used inside learning, not an AI technique by itself.

11 Natural Language Processing

Pipeline: tokenisation → stop-word removal → stemming / lemmatisation → POS tagging → parsing → semantics.

Word embeddings (Word2Vec, GloVe) map words to dense vectors where similar meanings lie close: **king − man + woman ≈ queen**.

Main challenge: ambiguity

- lexical: "bank" (river or money)
- syntactic: "I saw the man with the telescope"
- referential: "Rahim told Karim he passed": who?

Memory trick: **C**NN for **C**amera images, **R**NN for **R**ecurring sequences; stemming **chops** ("studies" → "studi"), lemmatisation **looks up** ("studies" → "study").

12 MCQs Asked: 49th BCS (ICT-281), 2025

Question (options)	Answer	Why
Not an AI technique? (gradient descent / genetic algorithm / deep learning / reinforcement learning)	**gradient descent**	a calculus optimiser; the rest are AI methods
ML model that uses backpropagation? (MLP / CNN / RNN / all)	**all of these**	RNN via BPTT
Best for time-series data? (CNN / RNN / DNN / ANN)	**RNN**	hidden state remembers the past
___ algorithm requires a policy (supervised / fuzzy / deep / reinforcement)	**reinforcement**	policy π: state → action
Main drawback of hill climbing? (memory / local optima / expands all / needs heuristic)	**local optima**	greedy, never goes downhill
Random search is ___ (greedy / local opt. / optimal / global optimisation)	**global optimisation**	samples the whole space
Not a regression metric? (R^2 / RMSE / Recall / MAE)	**Recall**	TP/(TP + FN): classification
Words → meaning vectors? (tokenisation / embeddings / stemming / POS)	**word embeddings**	Word2Vec, GloVe
Main challenge of NLP? (ambiguity / POS / syntax / tokenisation)	**ambiguity**	one sentence, many meanings

Regression: MAE = mean$|y - \hat{y}|$ · RMSE = $\sqrt{\text{mean}(y - \hat{y})^2}$ · $R^2 = 1 - SS_{res}/SS_{tot}$. Classification: accuracy, precision TP/(TP + FP), recall, **F1**.

13 Viva Rapid-Fire (bank IT vivas)

What is AI? → Machines doing tasks that need human intelligence: learning, reasoning, perception, language, decisions. Term: John McCarthy, Dartmouth **1956**.

What is ML? → AI that learns patterns from data and improves without explicit programming (shown many bottles, it recognises a new one).

What is deep learning? → ML with many-layer neural networks that learn features themselves.

ML vs DL? → ML: hand-made features, less data, explainable. DL: automatic features, big data + GPU, black box.

Normal vs super AI? → Narrow AI does one task (voice assistant, recommender): **all AI today**. AGI would match

Autonomous weapons? → Military systems that detect, select and engage targets with little or no human intervention.

How do they work? → Sensors (camera, radar, GPS) → AI/ML recognises the target → decision logic → actuators fire or steer. Issue: accountability, meaningful human control.

Robotics? → Designing, building and programming robots that sense, decide and act with minimal human help.

Robotics in banking? → Cash-sorting machines, document scanning, service robots and chatbots, and **RPA** bots for rule-based back-office work (KYC checks, reconciliation, reports).

humans; super AI would surpass them in every field: hypothetical.

Why Python for AI? → Simple syntax; libraries NumPy, pandas, scikit-learn, TensorFlow, PyTorch, Keras.

Night-Before-the-Exam Sheet

Formulas & facts:

A*: $f(n) = g(n) + h(n)$

Admissible: $h(n) \leq$ true cost

Consistent: $h(n) \leq c(n, n') + h(n')$

BFS: $O(b^d)$ time and space

$G = \Sigma\, \gamma^k\, r_{t+k+1}$

$Q \leftarrow Q + \alpha[r + \gamma \max Q' - Q]$

ε-greedy: random with prob. ε

$w \leftarrow w - \eta\, \partial L/\partial w$

$\text{conf}(X \rightarrow Y) = \text{sup}(X \cup Y)/\text{sup}(X)$

lift = conf/sup(Y), > 1 good

AdaBoost $\alpha = \frac{1}{2} \ln((1 - \varepsilon)/\varepsilon)$

3 voters at 0.7 → 0.784

One-liners

1. Supervised = labels; unsupervised = none; RL = reward
2. Policy → reinforcement learning
3. Agent = sensors + actuators
4. 5 agents: reflex, model, goal, utility, learning
5. Rational ≠ omniscient; task = PEAS
6. A* optimal iff h admissible (tree)
7. Hill climbing → local optima
8. Random search → global optimisation
9. High train, low test = overfitting
10. Bagging ↓ variance; boosting ↓ bias
11. Weak learner ≈ just above random
12. Market basket → Apriori, FP-Growth
13. Backprop: MLP, CNN, RNN (BPTT)
14. Time series → RNN; images → CNN

Top traps: RL is not unsupervised · A* is optimal only with admissible h · greedy best-first is not optimal · BFS is optimal only for equal step costs · gradient descent is an optimiser · confidence without lift misleads · agentic AI ≠ AGI · all AI today is narrow AI

All tricks together: Teacher / no teacher / critic · each agent type adds one thing · g = gone so far, h = how far still · a rare set makes bigger sets rare · a chatbot talks, an agent does · CNN = camera, RNN = recurring



the agent ladder: each step adds one ability

Written tips: "Differentiate" → a 6–8 row table with one example per column. "Explain" → definition, diagram, properties, example. A 10-mark "argue" → numbered premises and a one-line conclusion.

✓ Practice MCQs (from the asked topics)

1. Learning from labelled input–output pairs is:
(a) reinforcement (b) unsupervised
(c) supervised (d) clustering

2. An agent acting only on the current percept with if-then rules is:
(a) simple reflex (b) goal-based
(c) utility-based (d) learning

3. A* (tree search) is optimal when h(n) is:
(a) random (b) larger than the true cost
(c) negative (d) admissible

4. High training but low test accuracy means:
(a) underfitting (b) overfitting
(c) high bias (d) clean data

5. Random Forest is an example of:
(a) bagging (b) boosting
(c) stacking (d) clustering

6. sup({A, B}) = 0.4, sup({A}) = 0.5. conf(A → B) =
(a) 0.2 (b) 0.4
(c) 0.8 (d) 0.9

7. Best network for stock-price time series:
(a) CNN (b) decision stump
(c) k-means (d) LSTM

8. In ε-greedy, ε is the probability of:
(a) exploiting (b) exploring at random
(c) stopping (d) discounting

9. A classification (not regression) metric:
(a) F1 score (b) MAE
(c) RMSE (d) R^2

10. What most separates agentic AI from a chatbot?
(a) text answers (b) using an LLM
(c) planning and acting via tools (d) many languages

Answer key → 1-c · 2-a · 3-d · 4-b · 5-a · 6-c · 7-d · 8-b · 9-a · 10-c

Written question: Define a rational agent. Describe the five types of agent and say which type a self-driving taxi should be, and why. 10 marks

1. Rational agent: chooses the action expected to maximise its performance measure from its percepts and knowledge; rational ≠ omniscient. 2
2. Five types, one line each: simple reflex (rules on current percept), model-based (internal state), goal-based (search/plan), utility-based (max expected utility), learning (critic + learning element). 5
3. Taxi = **utility-based learning agent**: partially observable, stochastic roads need a model; it trades off speed, safety, comfort, fuel (utility) and improves with experience. 2
4. Close with its PEAS line. 1

Before the exam ✓

- ✓ Supervised / unsupervised / RL table
- ✓ Agent diagram + 5 agent types
- ✓ A* vs BFS argument in steps
- ✓ Apriori on 5 baskets with lift

Also practise:
① Agentic AI vs chatbot
② Overfitting: causes, fixes
③ Bagging vs boosting